

Deploying LiteLLM Proxy

GitHub Codespaces and Google Colab

Detailed Notes – Agentic AI Bootcamp

Contents

1	Introduction	2
2	Deployment Option 1: GitHub Codespaces	2
2.1	Step 1: Initialize the Environment	2
2.2	Step 2: Install LiteLLM	2
2.3	Step 3: Configuration	2
2.4	Step 4: Run and Expose the Proxy	3
3	Deployment Option 2: GitHub Codespaces using Docker Compose	3
3.1	Step 1: Create the Configuration Files	3
3.2	Step 2: Define Environment Variables	3
3.3	Step 3: Create the Docker Compose File	3
3.4	Step 4: Launch and Expose the Service	4
4	Deployment Option 3: Google Colab	4
4.1	Step 1: Setup and Installation	4
4.2	Step 2: Configure Environment and YAML	4
4.3	Step 3: Start Proxy and Tunnel	5
5	Reference Tutorials	5

1 Introduction

LiteLLM provides a fast, lightweight proxy server that allows you to call over 100+ LLM APIs using the standard OpenAI format. It manages authentication, load balancing, and fallbacks. This guide covers two alternative environments for quickly spinning up a LiteLLM Proxy for development and testing: GitHub Codespaces and Google Colab.

2 Deployment Option 1: GitHub Codespaces

GitHub Codespaces provides a complete, cloud-hosted Linux development environment directly accessible from your browser, making it an excellent choice for running the proxy alongside other workflow tools.

2.1 Step 1: Initialize the Environment

1. Open a new or existing repository on GitHub.
2. Click the **Code** button, switch to the **Codespaces** tab, and click **Create codespace on main**.
3. Once the terminal is available, update the package list and install Python pip if necessary:

```
sudo apt-get update
sudo apt-get install python3-pip
```

2.2 Step 2: Install LiteLLM

Install the LiteLLM package with the proxy dependencies:

```
pip install 'litellm[proxy]'
```

2.3 Step 3: Configuration

Create a `config.yaml` file in your workspace root to define your models and API keys:

```
model_list:
- model_name: llama-3.3-70b-versatile
  litellm_params:
    model: groq/llama-3.3-70b-versatile
    api_key: os.ENV/GROQ_API_KEY
```

Ensure you export the required API keys in your terminal:

```
export GROQ_API_KEY="gsk_YOUR_KEY_HERE"
```

2.4 Step 4: Run and Expose the Proxy

Start the proxy server:

```
litellm --config config.yaml --port 4000
```

Navigate to the **Ports** tab in your Codespace. Locate port 4000, right-click the visibility setting, and change it from **Private** to **Public**. You can now use the forwarded URL as your standard OpenAI API Base URL.

3 Deployment Option 2: GitHub Codespaces using Docker Compose

While running LiteLLM directly via Python is useful, deploying it as a containerized service using Docker Compose closely mirrors production environments and easily integrates alongside other workflow orchestration containers.

3.1 Step 1: Create the Configuration Files

First, define your model routing and observability callbacks in a `litellm.config.yaml` file at the root of your workspace:

```
model_list:
  - model_name: llama-3.3-70b-versatile
    litellm_params:
      model: groq/llama-3.3-70b-versatile
      api_key: os.ENV/GROQ_API_KEY

litellm_settings:
  success_callbacks: ["langfuse"]
  failure_callbacks: ["langfuse"]
```

3.2 Step 2: Define Environment Variables

Create an `.env` file to store your credentials securely. This ensures sensitive keys are passed safely into the container environment:

```
GROQ_API_KEY=gsk_YOUR_GROQ_KEY_HERE
LANGFUSE_PUBLIC_KEY=pk-1f-YOUR_PUBLIC_KEY_HERE
LANGFUSE_SECRET_KEY=sk-1f-YOUR_SECRET_KEY_HERE
LANGFUSE_HOST=https://jp.cloud.langfuse.com
```

3.3 Step 3: Create the Docker Compose File

Create a `docker-compose.yml` file that pulls the official LiteLLM image, mounts your configuration, and loads the environment variables:

```
version: '3.8'
services:
  litellm_proxy:
```

```
image: ghcr.io/berriai/litellm:main-latest
container_name: litellm_proxy
ports:
  - "4000:4000"
volumes:
  - ./litellm_config.yaml:/app/config.yaml
env_file:
  - .env
command: [ "--config", "/app/config.yaml", "--port", "4000" ]
```

3.4 Step 4: Launch and Expose the Service

1. Start the container stack in detached mode by running:

```
docker-compose up -d
```

2. Verify the container is running and check the initial startup logs:

```
docker logs -f litellm_proxy
```

3. In your Codespace, navigate to the **Ports** tab at the bottom panel. Locate port 4000, right-click under the **Visibility** column, and change it from **Private** to **Public**.
4. Copy the forwarded URL provided by GitHub. This URL acts as your standard OpenAI API Base URL for connecting agentic nodes.

4 Deployment Option 3: Google Colab

Google Colab offers a free Jupyter Notebook environment. While primarily used for interactive execution, you can use localtunnel or ngrok to expose a background LiteLLM proxy process to the public internet.

4.1 Step 1: Setup and Installation

Open a new Google Colab notebook and run the following in a cell to install LiteLLM and localtunnel:

```
!pip install 'litellm[proxy]'
!npm install -g localtunnel
```

4.2 Step 2: Configure Environment and YAML

Write the configuration to the Colab filesystem and set environment variables:

```
import os
os.environ["GROQ_API_KEY"] = "gsk_YOUR_KEY_HERE"

yaml_content = """
model_list:
  - model_name: gpt-3.5-turbo
    litellm_params:
      model: groq/llama-3.3-70b-versatile
      api_key: os.ENV/GROQ_API_KEY
"""

with open("config.yaml", "w") as f:
    f.write(yaml_content)
```

4.3 Step 3: Start Proxy and Tunnel

Run the proxy in the background using `nohup` and expose it using `localtunnel`:

```
!nohup litellm --config config.yaml --port 4000 > proxy.log 2>&1 &
!lt --port 4000
```

The output will provide a public `localtunnel` URL. You must click the provided link once to bypass the `localtunnel` warning screen, after which the URL acts as your public API base.

5 Reference Tutorials

Below are essential reference materials for further expanding your LiteLLM architecture:

- Quick Start:**
https://docs.litellm.ai/docs/proxy/docker_quick_start
Summary: A rapid guide to deploying the LiteLLM proxy locally using Docker containers.
- Hand's On Tutorial:**
https://colab.research.google.com/github/BerriAI/litellm/blob/main/cookbook/liteLLM_Getting_Started.ipynb
Summary: An interactive Jupyter/Colab notebook demonstrating the fundamentals of making unified API calls with LiteLLM.
- GitHub Copilot Integration:**
https://docs.litellm.ai/docs/tutorials/github_copilot_integration
Summary: Instructions on securely routing GitHub Copilot traffic through your custom LiteLLM proxy setup.
- OpenTelemetry (OTEL) v2:**
https://docs.litellm.ai/docs/observability/opentelemetry_v2
Summary: Documentation for setting up standardized OTEL tracing and metrics for robust observability.

- **Langfuse Integration:**

https://docs.litellm.ai/docs/observability/langfuse_integration

Summary: Step-by-step configuration for sending LiteLLM generations and traces directly to Langfuse.